Real Life Rail

A live train tracker for your Android phone. It uses Realtime Trains' record of every junction and timing point a train passes, so it shows where your train really is between stations, to the quarter minute on your Hobbyist plan.

Build report · Saturday 7 November 2026 · built and tested while you were at the theatre

What you need to do (about 5 minutes)

1. **Bring the work onto main** (it's on a branch, and I don't merge without you):

```
cd ~/Projects/realliferail
git merge --ff-only worktree-first-build
uv sync
```

2. **Set up the phone link** (no sudo needed). It prints a Tailscale sign-in link, then your app's address:

```
./scripts/phone-setup.sh
```

3. **On the phone:** install **Tailscale** from the Play Store, sign in with the same account, open the printed `https://realliferail.….ts.net` address in Chrome, then ⋮ → **Install app**.

Your Hobbyist key is already saved on the PC (`~/.config/realliferail/settings.json`, readable only by you), so the app starts in live mode. You can change or remove it in the app's **Settings**.

19	164	89	2
commits on the branch	unit and web tests pass	browser tests pass (phone and desktop, light and dark)	live tests pass against the real API

What it does

Find your train

- Type a station (instant suggestions from 2,609 stations, offline) and optionally a time, day and "calling at" station.
- The board looks like a departure board: big times, destination, live status, and a **platform plate** that's solid once confirmed and outlined while only expected.
- Empty stock and staff-only moves are hidden (detailed mode is full of them). "Show all" brings them back. Bus replacements are labelled.
- A "now" board updates itself every minute in place, without jumping to the top.

Track it

- A big countdown: **"2 min until it leaves Reading · 15:48 · platform 11"**, then where it's going and when it arrives.
- Every stop *and* every timing point it passes (a real GWR train today had **70**), with booked, reported and expected times.

- A **yellow train-front panel** moves along the line every second at the booked speed for that section. It holds short of the next point until that point is reported.
- It says plainly when a train goes quiet: “No report at New Malden yet, 3 min after expected. It may be held at a signal.” Later times then move back, in order, and are labelled as our estimate.
- “Lost most time between Barnwood Jn and Horton Road Jn (2 min)”: a station-only app can't tell you this.

Commute

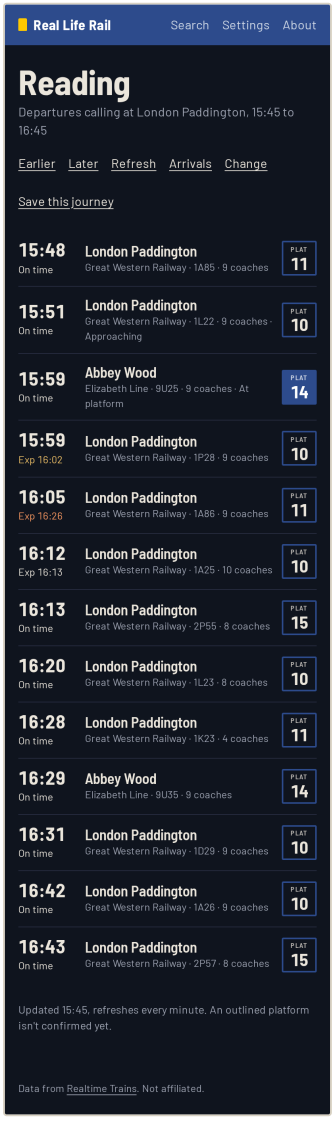
- **Save this journey** on any board with “calling at” set. The home screen then gives you one-tap **Reading → London Paddington** and **Return**.
- **Carry on tracking**: reopen the app and the train you were following is at the top.
- Coming back to the app after a while catches up at once.

Settings and plans

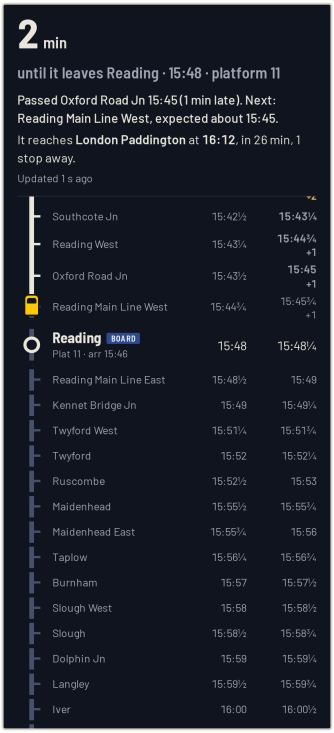
- Paste a key in Settings. It's checked with Realtime Trains first, kept only on the PC, and only ever shown as its last 5 characters.
- The app detects your plan from what the key is actually allowed, and turns features on to match. Free users work fully, just with less precision.
- Settings can only be changed from your PC, your home network or your own Tailscale devices.

Screenshots: real trains, today

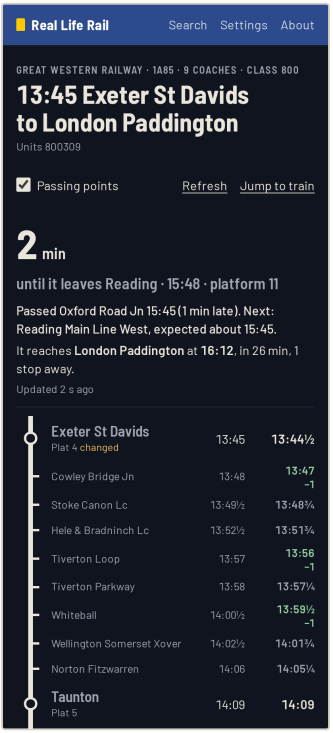
Taken with your Hobbyist key at 15:45 on the server running on your PC. This is live Realtime Trains data, not the simulator.



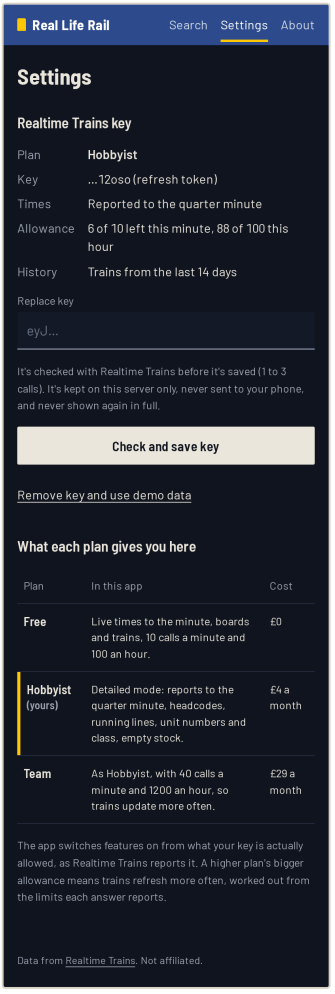
Reading, departures calling at London Paddington. Headcodes, coach counts, platform plates, "Approaching" and "At platform".



The countdown, then the line: the train (yellow) just past Reading Main Line West, quarter-minute times.

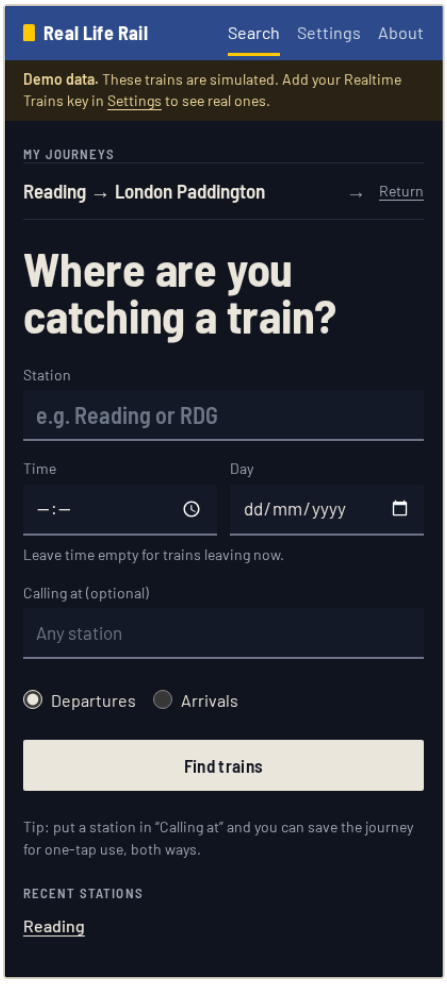


13:45 Exeter St Davids to Paddington. Class 800, unit 800309, "Plat 4 changed", early running in green.

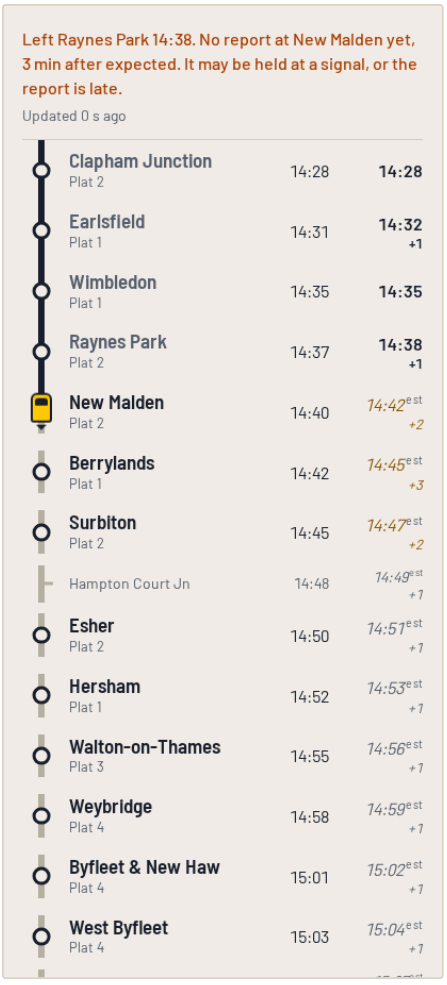


Settings: Hobbyist detected, key shown only as …12oso, live allowance.

Other states (from the test simulator)



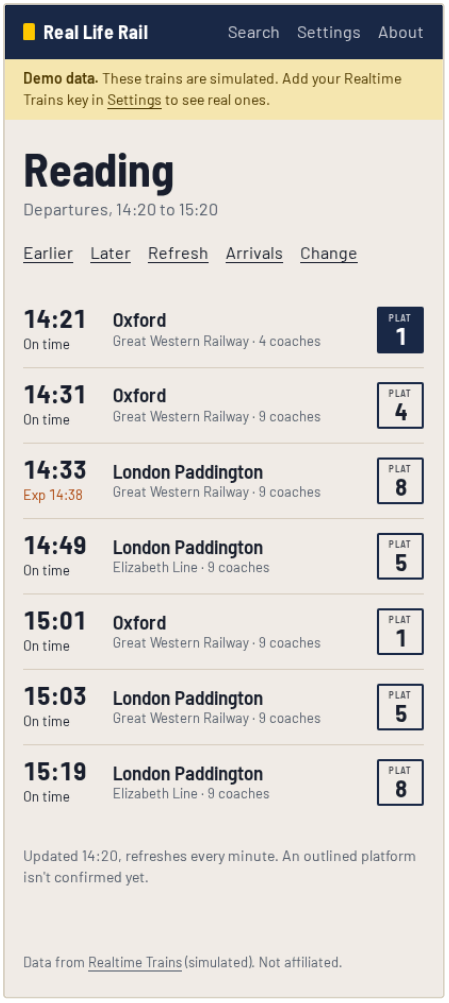
Home: saved journey with one-tap Return, search, recent stations.



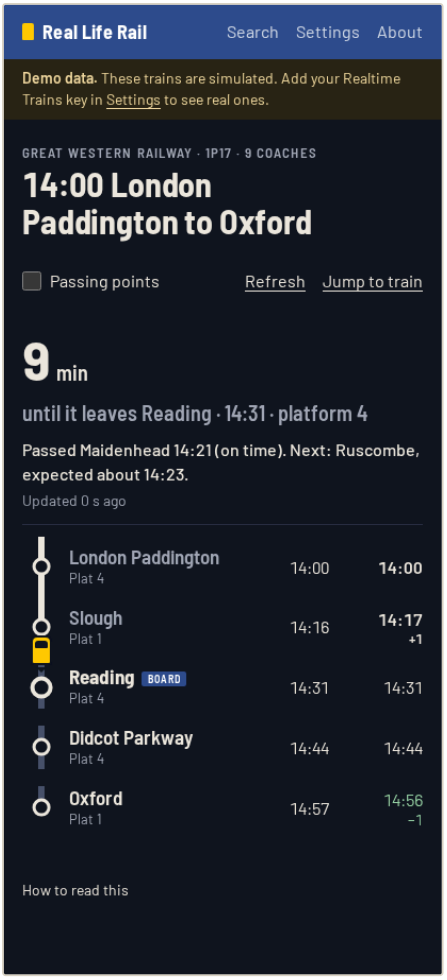
A train held between stations: said plainly, and every later time moved back as our estimate (italic).



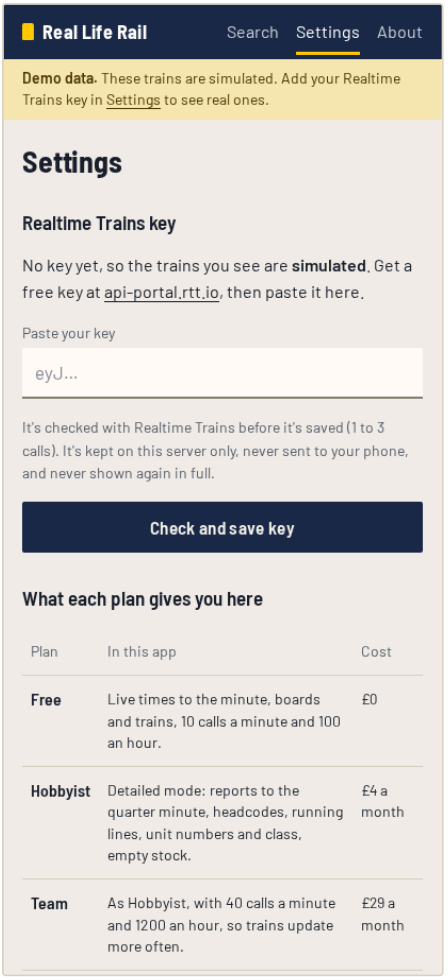
Not started yet: the train waits at its origin, with the countdown to your stop.



Board in daylight. Outlined plates are expected platforms.



Passing points hidden: stops only, and the train still placed by time.



Settings with no key: demo mode, and what each plan adds.

Using it on Android every day

The app runs on your PC and your phone opens it. To use it on the train, away from home Wi-Fi, the phone needs a safe way to reach the PC. **Tailscale** (free for personal use) makes a private network of just your own devices, and gives the app a real HTTPS address, so Chrome installs it as a proper app with its own icon and window. Nothing is opened to the internet.

- `scripts/phone-setup.sh` installs two **systemd user services**: the app, and Tailscale in no-root userspace mode. They start with the PC and restart if they stop. Then it runs `tailscale serve`, which publishes the app only to your devices. I installed Tailscale with Homebrew and checked that it starts without root and gives a sign-in link. The one step left is your sign-in.
- The PC has to be on. Any always-on Linux box works the same way (a Raspberry Pi is ideal).
- Just at home without Tailscale: `uv run realliferail serve --lan` and open port 8765 once with `firewall-cmd` (needs sudo). Over plain http, Chrome only adds a shortcut, not a full app.

How it uses your 10 calls a minute

Situation	What happens
Watching a train	The page asks again about every 40 s: `3600 ÷ (calls left this hour − 10)`, never under 30 s. It slows down by itself as the hour runs low, pauses while the app is in the background, and catches up the moment you come back.
Pressing Refresh	Refetches only if the answer is over 15 s old. Otherwise it says "Already up to date (8 s old)" without any request. The server enforces the same rule, so pressing it 20 times costs **one** call (tested).
Phone and PC on the same train	They share one answer. Concurrent requests for the same train are joined into one call (tested with 8 at once).
Searching	Boards are kept for a minute. The last 10 calls of each hour are kept for searches and never spent on background updates.
Allowance runs out, or RTT says slow down	The last answer is shown, marked as stale, or a "Realtime Trains is busy" page that retries every 30 s. It never claims the train doesn't exist. `Retry-After` is obeyed.
Restarting the server	Your refresh token's short-life access token is cached (0600), so a restart costs 0 calls instead of 3.

Plans: what changes in the app

	Free	Hobbyist (yours)	Team and up
Live times	to the minute	**to the quarter minute** (from signalling berth steps)	same
Passing points and the train between stations	yes	yes, more precise	same
Headcode, unit numbers, class, branding	—	yes	yes
Empty stock and staff-only moves	—	hidden, with "show all"	same
Update interval	about 40 s	about 40 s	faster, worked out from the limits RTT reports

The app decides from the key's entitlements and the limits each answer reports, not from plan names. Your key reports `allowDetailed` and `allowAllocationsPassenger`, 10 a minute, 100 an hour, 1,000 a day, and 14 days of history.

What I found in the real API with your key

- Your key is a **refresh token**: `/api/info` refuses it, and `/api/get_access_token` swaps it for a short-life access token. The app handles this itself.
- **Detailed mode gives actual times in quarter-minute steps** (`15:03:15`, `14:22:45`), on passing points as well as stops. RTT also **forecasts passing points**, and reports running lines (`UML`, `URL`), stock branding (Arterio, Voyager), unit numbers and class.
- A long-distance service had 56 points, 53 of them passes. Reading had 57 board rows in an hour, including 2 empty-stock moves and several bus rows.
- The real data mostly matches RTT's published spec. One mismatch: `stockType` is lowercase (`"unit"`) where the spec says `"UNIT"`. It's recorded and tolerated, not hidden.
- Three real responses are saved as test fixtures. The whole app is tested against them, as well as against the spec.
- **No signals.** RTT's API stops at timing points (stations, junctions, named passes). Signal-by-signal positions would need Network Rail's own train describer feed and a separate free account. That's a possible later stage, not something to fake.

The new look

You said the first look was boxy, AI-looking and too like your other apps. The new one comes from British railway design that people actually made and use:

- **British Rail's 1965 identity and Rail Alphabet**: one clear signage typeface, navy and white sign plates. Rail Alphabet is licensed, so **Barlow** (open licence, drawn from road and rail signs) stands in, with its condensed cut so long station names fit.
- **Network Rail wayfinding**: platform numbers on solid plates.
- **Line maps on carriage walls**: a thick line with ringed stops, track already covered in ink, passing points as small ticks.
- **The yellow warning panel on every British train front** is the train marker, windscreen included. It's also the focus ring and the logo mark.
- **Transit-style apps**: one giant number you can read while walking.
- **Printed timetables**: warm paper and navy ink instead of grey, and deep navy at night.

No cards, boxes, gradients, glow or pills. Contrast (4.5:1 for text) and 44 px tap targets are checked by tests.

Testing, and bugs it caught

Tests never touch the network, except two opt-in live tests that cost about 4 calls. The browser suite drives Chromium at 360 px and 412 px phone widths and on desktop, in light and dark, and saves a contact sheet. I read it after every visual change. Bugs found and fixed this session:

- A held train still showed RTT forecasts that were already impossible: two consecutive stops at the same minute, or times in the past. These are now replaced by a labelled estimate based on the fastest the train could physically run.
- "Overdue" could never fire once estimates were pushed back. It now compares against the time expected before the hold.
- Running out of allowance showed "train not found" (404). It now shows a 503 "busy" page that retries.
- The demo simulator locked itself out after 100 calls. Its rate-limit windows now slide.
- The train marker was placed before the web font loaded, so it was briefly in the wrong spot.
- The travelled line regrew from the top on every refresh.
- With passing points hidden, the marker sat in the wrong place.
- The nav links (37 px) and the new key field (27 px) were too small to tap.
- "padd" suggested Paddock Wood before Paddington.
- "Port in use" crashed silently. Your mailman app is on 8767, so it now says so in a sentence.

- Our own estimates showed false quarter-minute precision. They now round to the minute.

Plainly: limits and what's not done

- **How much time a late train wins back is a guess** (75% of the recovery time booked in each section). It's labelled "est" wherever it's used. `realliferail record` collects what's needed to tune it against reality.
- Tailscale needs your sign-in, so I couldn't finish the phone link. The script and services are tested up to that point.
- The systemd services expect the code at `~/Projects/realliferail`, which is why step 1 (the merge) comes first.
- Nothing has been pushed to GitHub. That's left for you to ask for.
- A copy of the key is also in the worktree's `.env`, which git ignores. If you delete the worktree, the copy in `~/.config/realliferail` is still there.
- Not tried: running the server on the phone itself (Termux), because the app needs Python 3.14.

Where things are

Code	`~/Projects/realliferail/.claude/worktrees/first-build`, branch `worktree-first-build` (19 commits; `main` has the plan)
Standing brief	`CLAUDE.md`, based on uppYT's: rules, architecture, product rules, things that caught us out
Why things are as they are	`docs/decisions.md` · API reference with observations: `docs/rtt-api.md` · design references: `docs/design.md`
Your key	`~/.config/realliferail/settings.json` (0600). Cached access token: `access.json`
Commands	`uv run realliferail serve` · `board RDG --to PAD` · `train ID` · `token` · `record`
Tests	`uv run pytest` · `uv run pytest -m ui` (then `.ui-report/index.html`) · `uv run pytest -m live -n 0`